Arquitectura del Sistema de Banca por Internet BP

1. Introducción

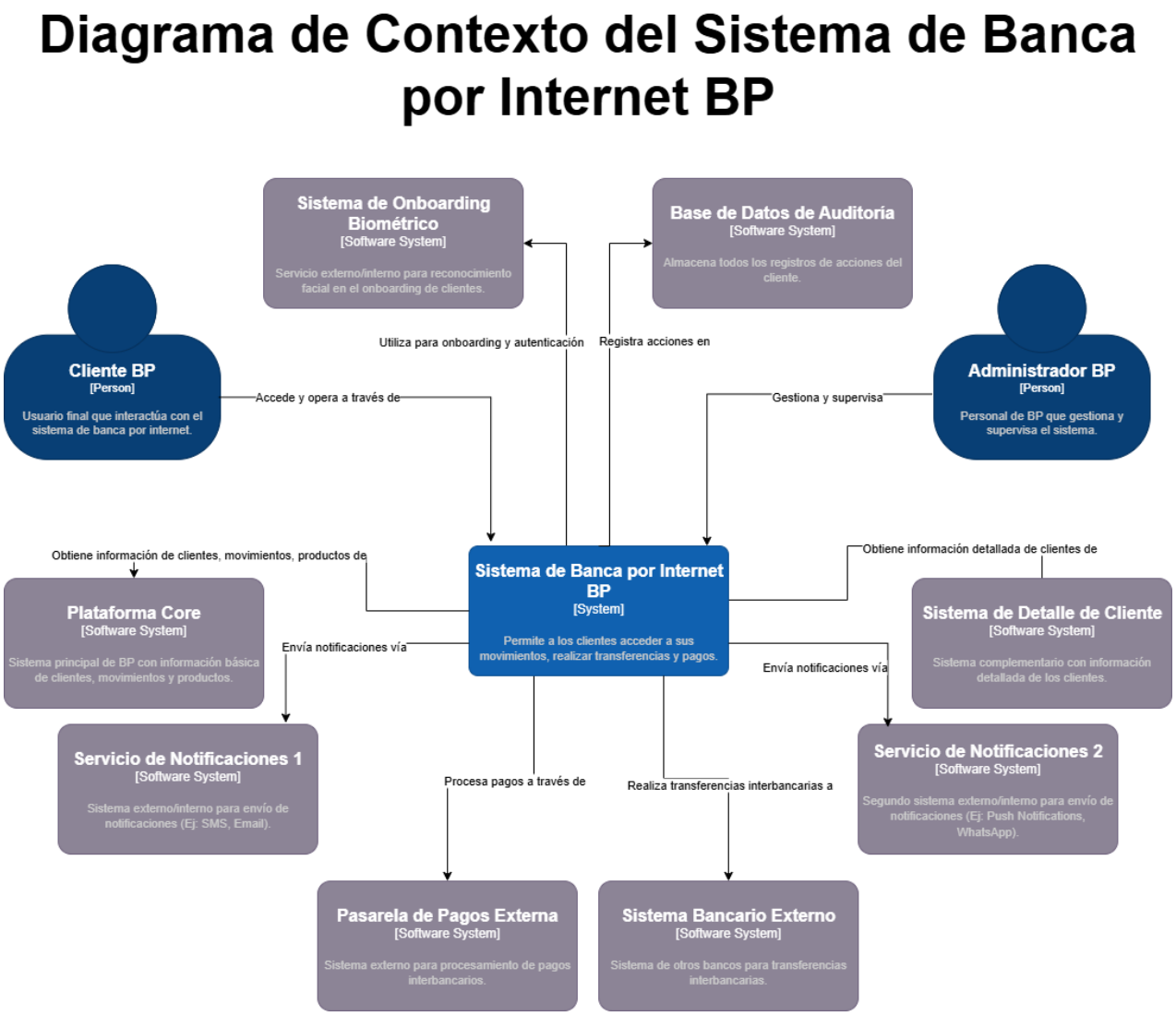
El presente documento describe la arquitectura propuesta para el nuevo sistema de banca por internet de BP. El objetivo es proporcionar una plataforma robusta y segura que permita a los usuarios acceder al historial de movimientos, realizar transferencias y pagos entre cuentas propias e interbancarias, garantizando la confidencialidad, integridad y disponibilidad de la información.

La arquitectura se fundamenta en los siguientes principios:

- Microservicios: Para agilidad, escalabilidad independiente y resiliencia.
- Diseño API-First: Para desacoplamiento y extensibilidad.
- Seguridad por Diseño: Integrada en cada capa del sistema.
- Observabilidad: Monitoreo y trazabilidad completos.
- Resiliencia: Alta disponibilidad, tolerancia a fallos y recuperación ante desastres.

2. Diagrama de Contexto (C4 Model - Nivel 1)

A continuación, se presenta el Diagrama de Contexto, que ofrece una vista de alto nivel del sistema, sus usuarios y las interacciones con otros sistemas externos.

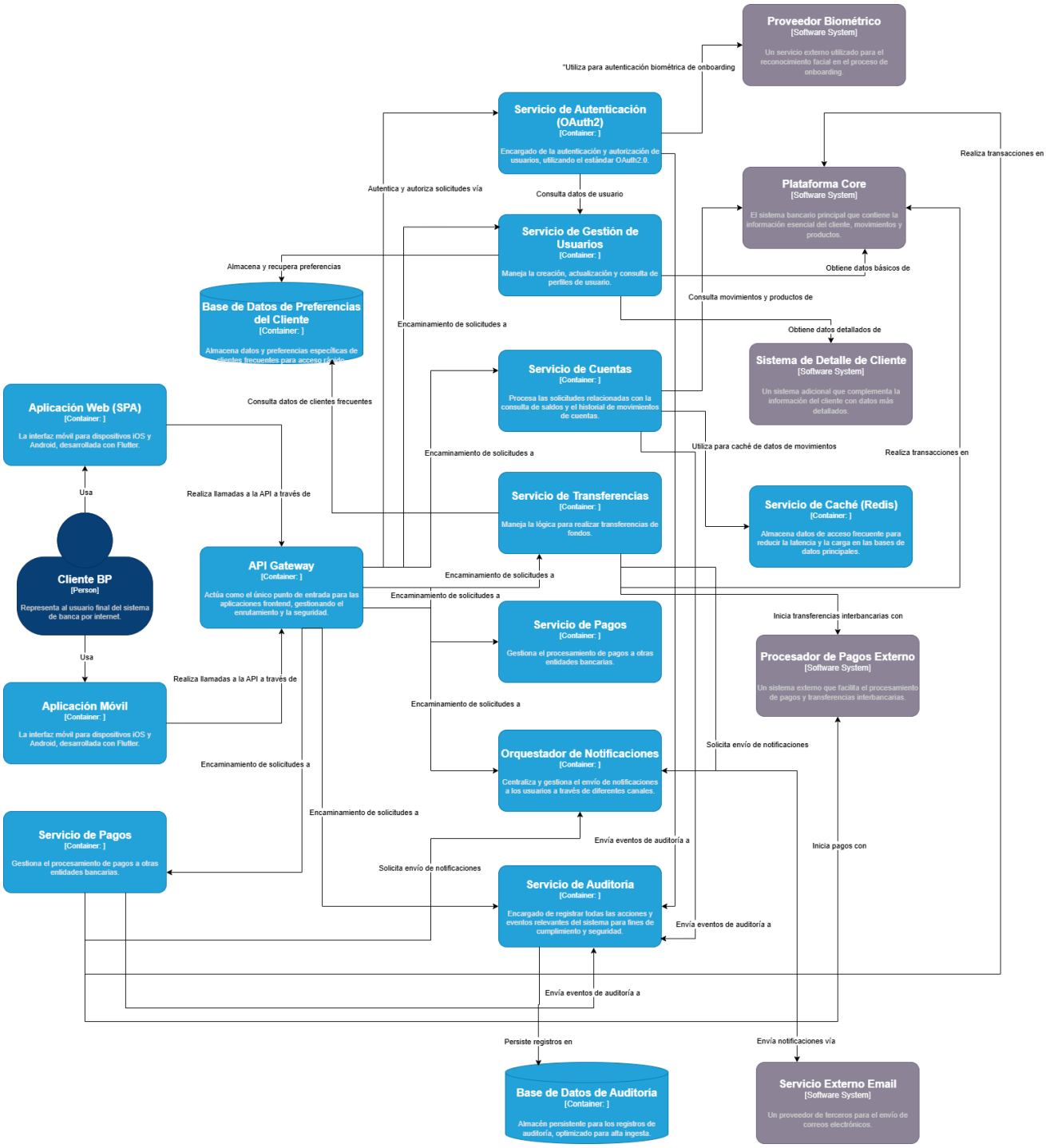


Explicación del Diagrama de Contexto:

Este diagrama muestra cómo el sistema interactúa con sus principales usuarios (Cliente BP, Administrador BP) y los sistemas externos o propios de la entidad. Se destacan las dependencias con la plataforma core y el sistema de detalle de cliente para obtener la información del cliente.También se visualizan las interacciones con sistemas de pago y transferencias interbancarias, el sistema de onboarding biométrico y la base de datos de auditoría.

3. Diagrama de Contenedores (C4 Model - Nivel 2)

El Diagrama de Contenedores descompone el sistema en unidades más grandes y ejecutables, como aplicaciones, servicios y bases de datos.



Explicación del Diagrama de Contenedores:

Este diagrama desglosa el sistema en sus componentes de alto nivel. Se identifican las Aplicaciones Frontend (SPA y Móvil), un API Gateway como punto de entrada único, y varios microservicios para funcionalidades específicas como autenticación, gestión de usuarios, cuentas, transferencias, pagos, orquestación de notificaciones y auditoría. También servicios de Caché (Redis) para optimizar el rendimiento.

Justificaciones de Decisiones:

Microservicios:

- Justificación 1 (Escalabilidad y Agilidad): La arquitectura de microservicios permite el desarrollo, despliegue y escalado independiente de cada servicio.
- Justificación 2 (Resiliencia y Tolerancia a Fallos): El aislamiento de los servicios significa que un fallo en un componente no necesariamente afectará a todo el sistema.

API Gateway (AWS API Gateway / Azure API Management):

- Justificación 1 (Seguridad y Gestión de Tráfico): Un API Gateway centraliza la gestión de seguridad (autenticación, autorización, validación de esquemas), la limitación de tasas, el enrutamiento y la transformación de solicitudes.
- Justificación 2 (Desacoplamiento Frontend/Backend y Observabilidad): El API Gateway desacopla los clientes frontend de la complejidad de los microservicios backend, permitiendo que estos evolucionen independientemente.

Servicio de Autenticación (OAuth2.0):

- Justificación 1 (Seguridad Estándar y Cumplimiento): Utilizar un proveedor de identidad de terceros como OAuth2.0, garantiza el cumplimiento de los estándares de seguridad de la industria.
- Justificación 2 (Experiencia de Usuario y Escalabilidad): Estos servicios ofrecen características avanzadas como inicio de sesión único y autenticación multifactor (MFA).

Bases de Datos Separadas (Auditoría, Preferencias, Core/Detalle):

- Justificación 1 (Optimización de Rendimiento y Escalabilidad): Separar las bases de datos por su propósito (ej. transaccional, analítica, auditoría) permite optimizar el rendimiento para cada caso de uso.
- Justificación 2 (Cumplimiento y Gobernanza de Datos): La segregación de datos facilita el cumplimiento de normativas específicas de auditoría y privacidad (GDPR, ISO 27001).

Opciones de Aplicaciones Frontend (SPA y Móvil):

Aplicación Web (SPA):

- Opción 1: React.js:
    - Justificación: React.js es un framework muy popular y robusto, respaldado por una gran comunidad. Ofrece un rendimiento excelente gracias a su Virtual DOM y un ecosistema maduro de librerías y herramientas.

- Opción 2: Angular:
    - Justificación: Angular es un framework completo mantenido por Google, que proporciona una estructura y convenciones claras, lo cual es beneficioso para grandes equipos y proyectos a largo plazo.

Aplicación Móvil Multiplataforma:

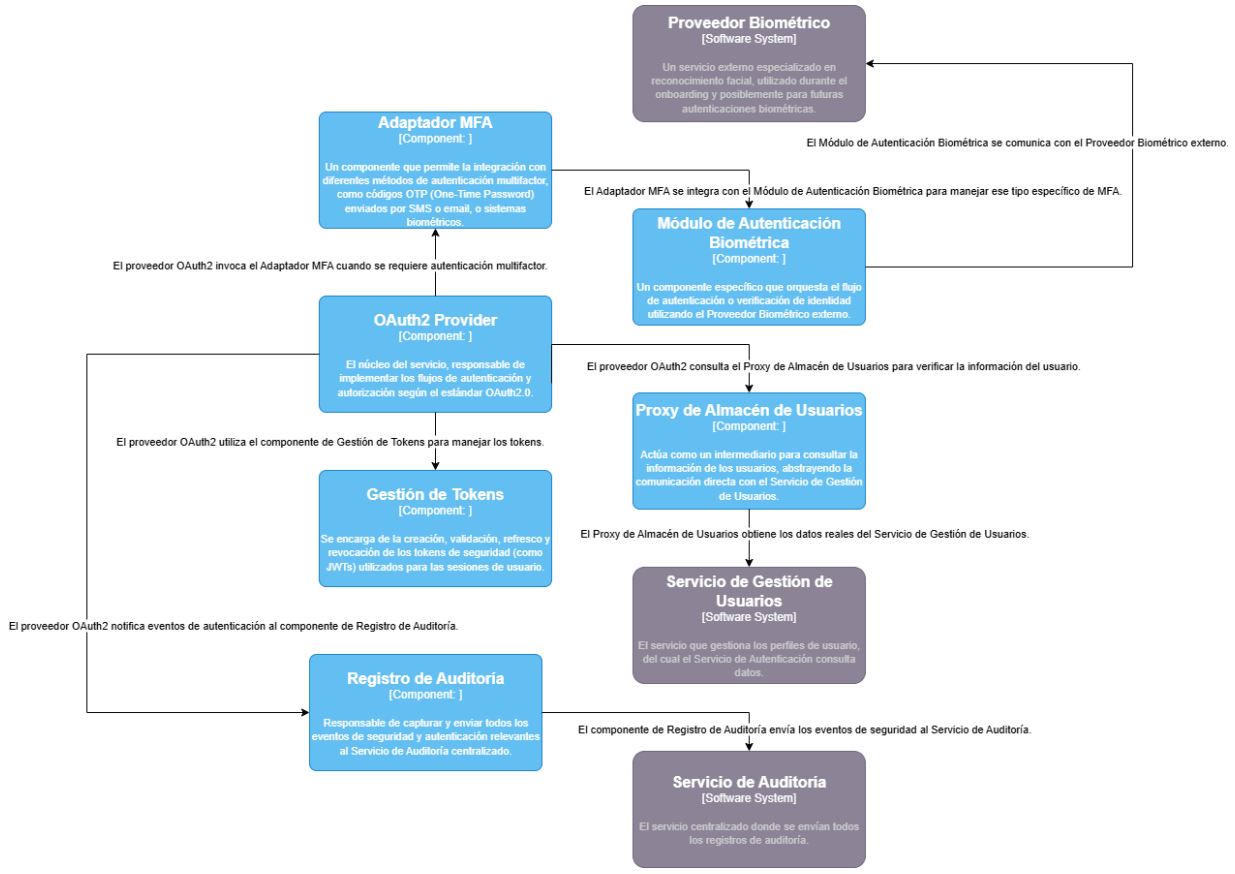
- Opción 1: Flutter:
  - Justificación: Flutter, desarrollado por Google, permite construir aplicaciones nativas compiladas a partir de una única base de código.

- Opción 2: React Native:
  - Justificación: React Native permite desarrollar aplicaciones móviles nativas utilizando JavaScript y React. Es una opción sólida si el equipo ya tiene experiencia con React.

4. Diagrama de Componentes (C4 Model - Nivel 3)

El Diagrama de Componentes proporciona un mayor nivel de detalle, mostrando los componentes internos de cada contenedor, sus patrones arquitectónicos y protocolos de comunicación.

4.1. Componentes del Servicio de Autenticación (auth_service)



Explicación y Justificaciones (Componentes del Servicio de Autenticación):

El Servicio de Autenticación es crucial para la seguridad. Aquí se detallan los componentes internos:

- OAuth2 Provider.
- Gestión de Tokens.
- Adaptador MFA.
- Proxy de Almacén de Usuarios.
- Módulo de Autenticación Biométrica.
- Registro de Auditoría.

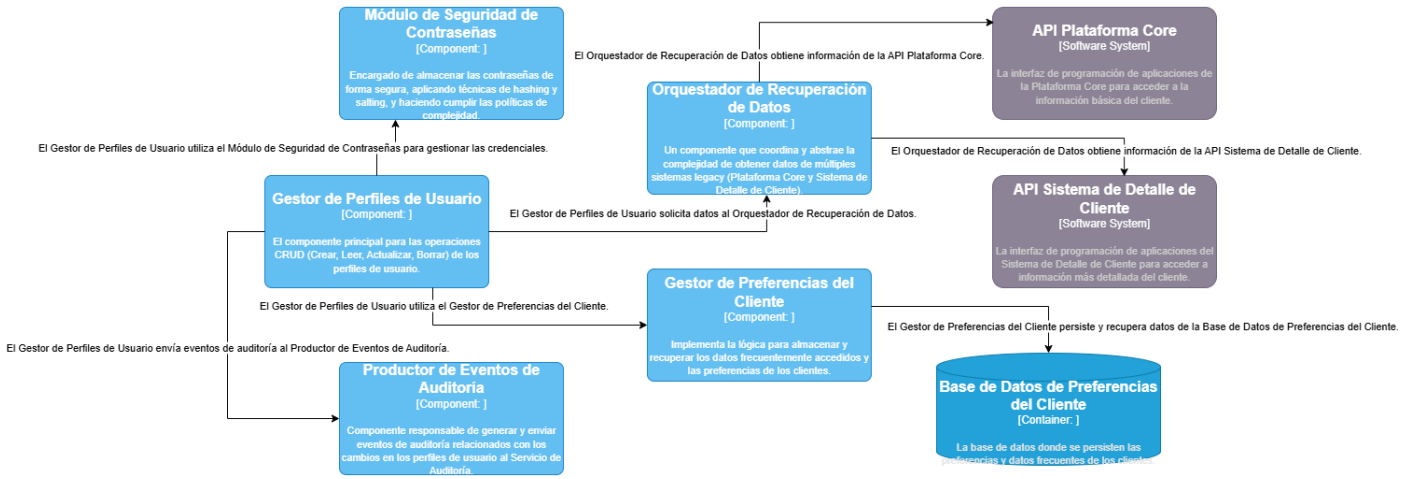
Flujo de Autenticación Recomendado (OAuth2.0):

- Justificación 1 (Seguridad Mejorada para Clientes Públicos): PKCE fue diseñado para clientes "públicos" (como SPAs y aplicaciones móviles) que no pueden almacenar de forma segura información sensible de un cliente.
- Justificación 2 (Cumplimiento y Estándar de la Industria): Este flujo es el más recomendado por la OAuth 2.0 Security Best Current Practice (BCP) para aplicaciones móviles y SPAs, proporcionando un alto nivel de seguridad para las credenciales del usuario y los tokens de acceso.

Flujo de Onboarding con Reconocimiento Facial:

- El usuario inicia el onboarding en la Aplicación Móvil.
- La aplicación solicita al usuario que complete el reconocimiento facial a través del Proveedor Biométrico.
- El Servicio de Autenticación interactúa con el Proveedor Biométrico para validar la identidad.
- Si el reconocimiento facial es exitoso, el Servicio de Autenticación utiliza el Servicio de Gestión de Usuarios para crear el nuevo perfil de usuario, incluyendo el registro de la clave de usuario.
- Una vez el onboarding ha finalizado, el usuario puede acceder al sistema usando usuario y contraseña.
- Huella Dactilar / Biometría del Dispositivo (FIDO2/WebAuthn): Se puede integrar un módulo de WebAuthn/FIDO2 en el Servicio de Autenticación.
    - Justificación: FIDO2 y WebAuthn son estándares de autenticación fuertes y resistentes al phishing que permiten el uso de autenticadores biométricos (como huellas dactilares o reconocimiento facial del dispositivo) y claves de seguridad.
    - Implementación: El usuario registra su huella/biometría con un autenticador compatible con FIDO2 durante el onboarding o en la configuración de seguridad. En futuros inicios de sesión, el autenticador verifica la biometría y genera una credencial criptográfica que se envía al servidor para verificación.).

4.2. Componentes del Servicio de Gestión de Usuarios (user_management_service)



Explicación y Justificaciones (Componentes del Servicio de Gestión de Usuarios):

El Servicio de Gestión de Usuarios centraliza la información del cliente.

- Gestor de Perfiles de Usuario: CRUD (Crear, Leer, Actualizar, Borrar) para perfiles.
- Módulo de Seguridad de Contraseñas: Asegura el almacenamiento de contraseñas con algoritmos fuertes como BCrypt o Argon2.

- Orquestador de Recuperación de Datos: Abstrae la complejidad de interactuar con la Plataforma Core y el Sistema de Detalle de Cliente.
- Gestor de Preferencias del Cliente: Implementa el patrón de diseño para la persistencia de datos de clientes frecuentes.
- Productor de Eventos de Auditoría: Envía todos los cambios relevantes de perfil al Servicio de Auditoría.

Patrón de Diseño para Persistencia de Información para Clientes Frecuentes:

Para la persistencia de información para clientes frecuentes, se propone el patrón Cache-Aside (Lectura y Escritura) en conjunto con una base de datos relacional (PostgreSQL) para las preferencias del cliente y Redis como servicio de caché.

- Justificación 1 (Rendimiento y Reducción de Carga en Sistemas Backend): El patrón Cache-Aside permite que los datos frecuentemente accedidos por clientes se almacenen en una caché de baja latencia (Redis).
- Justificación 2 (Resiliencia y Disponibilidad): En caso de indisponibilidad temporal de los sistemas backend de datos detallados o la Plataforma Core para ciertas consultas, el sistema aún puede servir información desde la caché, mejorando la experiencia del usuario y la resiliencia general del sistema para datos que no requieren consistencia instantánea.

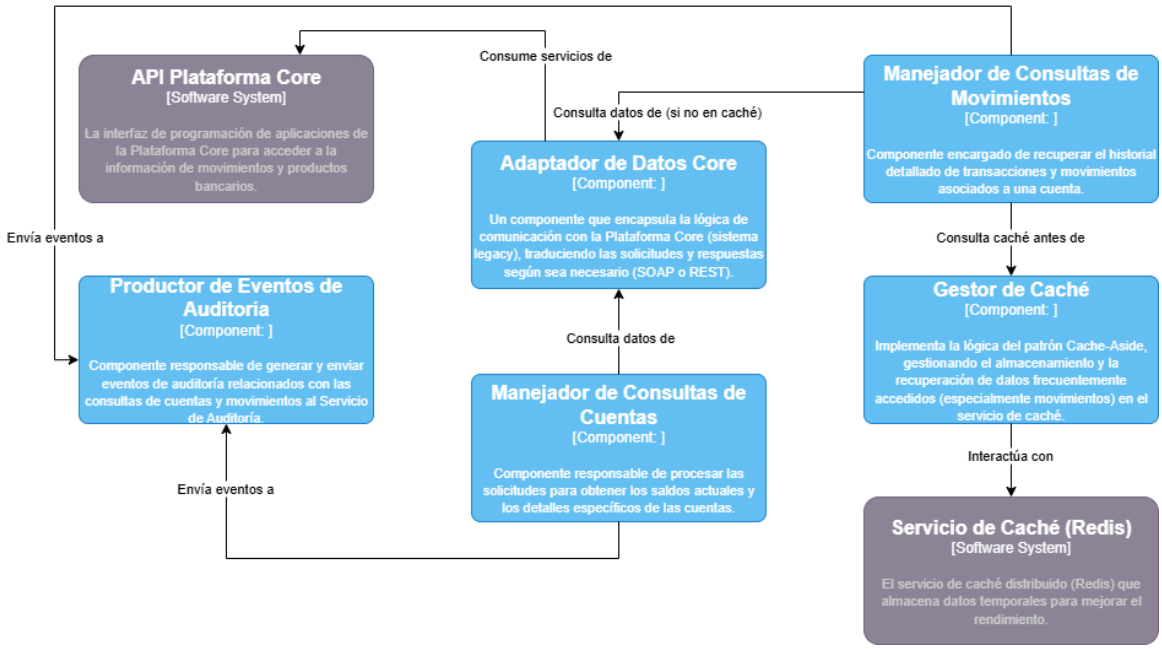
Interacción para Persistencia de Clientes Frecuentes:

1. Consulta de Datos (Lectura):
   - Cuando un servicio (ej. Servicio de Transferencias) necesita datos de un cliente frecuente, primero consulta el Servicio de Caché (Redis).
   - Si los datos están en caché (Cache Hit), se devuelven inmediatamente.
   - Si los datos no están en caché (Cache Miss), el Gestor de Preferencias del Cliente en el Servicio de Gestión de Usuarios consulta la Base de Datos de Preferencias del Cliente.
   - Una vez obtenidos de la DB, los datos se almacenan en el Servicio de Caché para futuras consultas y luego se devuelven al servicio solicitante.

2. Actualización de Datos (Escritura):
   - Cuando se actualizan los datos de un cliente frecuente el Gestor de Preferencias del Cliente primero actualiza la Base de Datos de Preferencias del Cliente.
   - Luego, el Gestor de Preferencias del Cliente invalida o actualiza los datos correspondientes en el Servicio de Caché para asegurar la consistencia.

4.3. Componentes del Servicio de Cuentas (accounts_service)

A continuación, se presenta el detalle de los componentes internos del Servicio de Cuentas, que se enfoca en la consulta de información financiera.



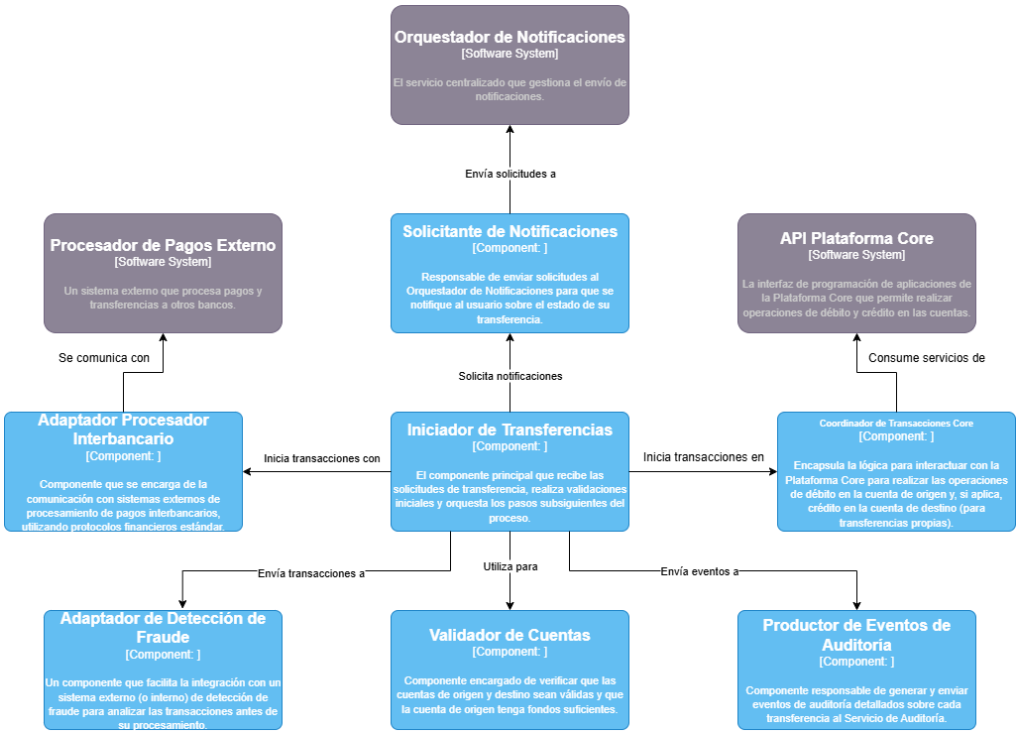
Explicación y Justificaciones (Componentes del Servicio de Cuentas):

El Servicio de Cuentas se enfoca en la consulta de información financiera.

- Manejador de Consultas de Cuentas/Movimientos: Componentes de lógica de negocio para las consultas.
- Adaptador de Datos Core: Encapsula la comunicación con la Plataforma Core, traduciendo formatos si es necesario.
- Gestor de Caché: Implementa el patrón Cache-Aside para el historial de movimientos.

4.4. Componentes del Servicio de Transferencias (transfer_service)

A continuación, se presenta el detalle de los componentes internos del Servicio de Transferencias, un componente crítico que maneja la lógica compleja de las transacciones monetarias.



Explicación y Justificaciones (Componentes del Servicio de Transferencias):

El Servicio de Transferencias es un componente importante que maneja la lógica compleja de las transacciones monetarias.

- Iniciador de Transferencias: Punto de entrada para las solicitudes.
- Validador de Cuentas: Asegura que las cuentas son válidas y tienen fondos suficientes.
- Adaptador de Detección de Fraude: Crucial para la seguridad, integra con sistemas externos de detección de anomalías.
- Coordinador de Transacciones Core: Gestiona la interacción con la Plataforma Core para las operaciones de débito/crédito.
- Adaptador Procesador Interbancario: Se comunica con sistemas externos utilizando protocolos financieros estándar (ISO 20022, SWIFT, ACH).
- Solicitante de Notificaciones: Asegura que se envíen las notificaciones obligatorias tras cada movimiento.

Servicios Adicionales Propuestos para Mejorar Rendimiento/Información:

1. Servicio de Personalización/Preferencias del Cliente:

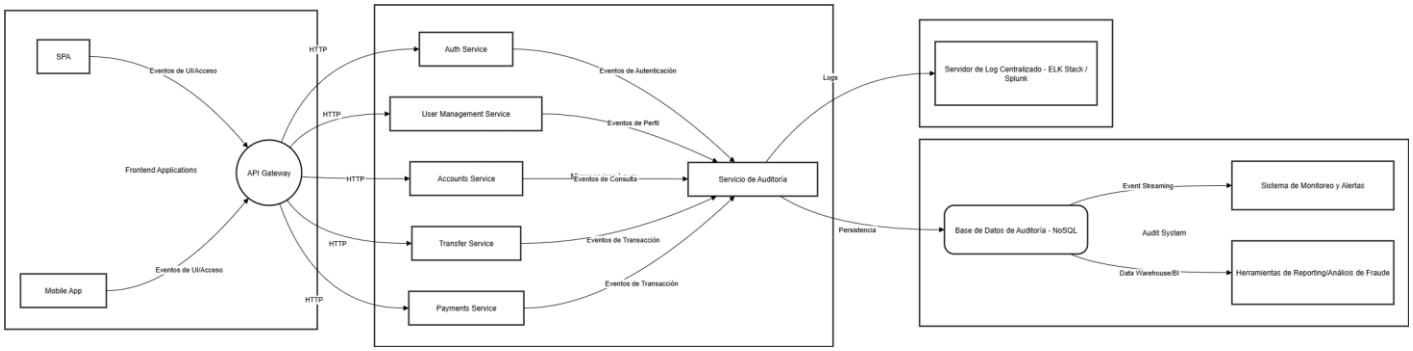
    - Justificación 1 (Mejora de la Experiencia de Usuario): Este servicio, ya conceptualizado en el Diagrama de Contenedores y el Servicio de Gestión de Usuarios, permitiría almacenar y gestionar preferencias explícitas del usuario. Esto proporciona una experiencia más adaptada, aumentando la satisfacción del cliente.
    - Justificación 2 (Optimización de Carga): Al almacenar estas preferencias en una base de datos optimizada para lectura rápida (como PostgreSQL con caching), se reduce la carga sobre los sistemas Core al no tener que consultarlos para cada pequeña preferencia del usuario.

2. Servicio de Límites y Tarifas:

    - Justificación 1 (Transparencia y Precisión): Un servicio dedicado a gestionar los límites transaccionales (diarios, mensuales) y las tarifas asociadas a diferentes tipos de transacciones (interbancarias, rápidas, etc.) garantiza que la lógica de negocio sea consistente y que los clientes reciban información precisa y en tiempo real.
    - Justificación 2 (Cumplimiento Normativo y Auditoría): Centralizar la lógica de límites y tarifas facilita el cumplimiento de las regulaciones financieras que a menudo imponen restricciones en las transacciones.

5. Diseño de Solución de Auditoría

El sistema de auditoría es un componente importante para el cumplimiento normativo, la seguridad y la trazabilidad de las operaciones en un entorno bancario. A continuación, se presenta su arquitectura.



Explicación y Justificaciones (Solución de Auditoría):

1. Modelo de Datos de Auditoría (Evento de Auditoría):

   - Un registro de auditoría debe contener información estandarizada para facilitar su análisis y cumplimiento. Se propone un esquema de evento de auditoría que incluya:
     - `timestamp`: Marca de tiempo de la ocurrencia del evento.
     - `correlationId`: ID único para trazar una secuencia de operaciones relacionadas.
     - `userId`: Identificador único del usuario que realizó la acción.
     - `userType`: Tipo de usuario (ej. Cliente, Administrador).
     - `sourceIp`: Dirección IP desde donde se originó la acción.
     - `serviceName`: Nombre del microservicio que generó el evento.
     - `eventType`: Tipo de evento (ej. "LOGIN_SUCCESS", "TRANSFER_INITIATED", "ACCOUNT_QUERY").
     - `status`: Estado de la operación (ej. "SUCCESS", "FAILURE").
     - `details`: Objeto JSON con detalles específicos del evento (ej. monto de transferencia, cuentas involucradas, error_code).
     - `securityContext`: Información de seguridad relevante (ej. tipo de autenticación, sesión ID).
   - Justificación 1 (Trazabilidad y Cumplimiento Normativo): Un esquema de datos estandarizado y detallado es esencial para reconstruir el historial de acciones de un usuario o de una transacción..
   - Justificación 2 (Análisis y Monitoreo): Un formato consistente facilita la obtención de datos en herramientas de análisis (SIEM, BI) y permite generar dashboards y alertas eficientes.

2. Mecanismo de Persistencia (Base de Datos de Auditoría - NoSQL):

   - Se recomienda una base de datos NoSQL orientada a documentos o clave-valor (ej., MongoDB, AWS DynamoDB, Azure Cosmos DB, Firebase) para la Base de Datos de Auditoría.
   - Justificación 1 (Escalabilidad Horizontal): Los sistemas de auditoría requieren manejar un alto volumen de eventos de forma continua y con baja latencia. Las bases de datos NoSQL son escalables horizontalmente y están diseñadas para cargas de escritura de alto volumen.
   - Justificación 2 (Flexibilidad de Esquema y Rendimiento): La naturaleza sin esquema (schemaless) de muchas NoSQL permite almacenar eventos de auditoría con detalles variables sin necesidad de migraciones de esquema complejas.

3. Componentes que Interactúan:

   - Aplicaciones Frontend (SPA, Móvil): Envían eventos de interacción de UI, inicio de sesión/cierre de sesión al API Gateway, que a su vez los pasa al Servicio de Auditoría.
   - API Gateway: Registra eventos de acceso a la API, intentos de autenticación fallidos, etc.

- Microservicios (Auth, User Management, Accounts, Transfer, Payments): Cada microservicio es responsable de enviar eventos de auditoría relevantes a través de un Productor de Eventos de Auditoría interno.
    - Justificación: Un Message Broker proporciona fiabilidad y durabilidad en la entrega de eventos, incluso si el Servicio de Auditoría o la Base de Datos de Auditoría están temporalmente no disponibles.

4. Sistema de Monitoreo y Análisis:

    - Los datos de la Base de Datos de Auditoría se pueden alimentar a un Sistema SIEM (Security Information and Event Management) como Splunk, ELK Stack (Elasticsearch, Logstash, Kibana) o herramientas de seguridad en la nube (AWS Security Hub, Azure Sentinel).
    - Justificación: Estos sistemas permiten la correlación de eventos, detección de anomalías, generación de alertas en tiempo real sobre actividades sospechosas y la creación de informes de cumplimiento.

6. Consideraciones Normativas y de Seguridad

El diseño de un sistema bancario por internet exige un estricto cumplimiento de diversas normativas y estándares de seguridad.

6.1. Normativas Clave

1. Ley de Protección de Datos Personales (ej. GDPR, LOPD en Ecuador):
    - Relevancia: Regulan la recopilación, procesamiento y almacenamiento de datos personales.
    - Impacto en la Arquitectura: Implica la implementación de políticas de anonimización/pseudonimización, cifrado de datos en reposo y en tránsito, control de acceso basado en roles (RBAC) y la capacidad de gestionar los derechos del interesado.

2. PCI DSS (Payment Card Industry Data Security Standard):

    - Relevancia: Si el sistema maneja directamente datos de tarjetas de crédito o débito, debe cumplir con este estándar para proteger la información de los titulares de tarjetas.
    - Impacto en la Arquitectura: Requiere segmentación de red, cifrado de datos sensibles, escaneos de vulnerabilidades regulares, monitoreo y registro de acceso, y políticas de seguridad robustas..

3. ISO 27001 (Sistema de Gestión de Seguridad de la Información):

    - Relevancia: Un estándar internacional para la gestión de la seguridad de la información. Proporciona un marco para establecer, implementar, mantener y mejorar un SGSI.
    - Impacto en la Arquitectura: Guía las prácticas de seguridad a lo largo de todo el ciclo de vida del desarrollo de software (SSDLC), incluyendo la gestión de riesgos, controles de acceso, gestión de incidentes, continuidad del negocio y auditorías.

4. Regulaciones Locales del Banco Central/Superintendencia de Bancos (ej. BCE en Ecuador):

    - Relevancia: Cada país tiene sus propias regulaciones específicas para el sector financiero, que pueden incluir requisitos sobre la seguridad de las transacciones electrónicas, la continuidad del negocio, la protección del consumidor y la infraestructura tecnológica.

- Impacto en la Arquitectura: Es fundamental revisar y adherirse a estas regulaciones para la aprobación y operación del sistema. Esto puede influir en la elección de proveedores de nube (si deben estar en el país), requisitos de redundancia y auditoría específicos.

6.2. Seguridad en la Arquitectura

1. Seguridad por Diseño (Security by Design):
    a. Principios: La seguridad debe ser una consideración fundamental desde las etapas iniciales del diseño. Esto incluye análisis de amenazas (ej. STRIDE), modelado de amenazas y la implementación de controles de seguridad en cada capa.
    b. Justificación: Es más costoso y menos efectivo añadir seguridad como un "parche" al final del ciclo de desarrollo. Integrar la seguridad desde el principio reduce vulnerabilidades y riesgos.

2. Cifrado de Datos (En Reposo y En Tránsito):
    a. En Reposo: Todas las bases de datos y almacenamiento de archivos deben cifrarse (ej. AWS KMS, Azure Key Vault con cifrado de disco).
    b. En Tránsito: Todas las comunicaciones internas y externas deben utilizar TLS 1.2+ (ej. HTTPS para APIs, TLS para bases de datos).
    c. Justificación: Protege la confidencialidad de los datos contra accesos no autorizados, tanto en caso de robo de dispositivos como de intercepción de comunicaciones.

3. Gestión de Identidad y Acceso (IAM):
    a. Componentes: Uso de un Servicio de Autenticación (OAuth2), implementación de RBAC (Role-Based Access Control) en todos los microservicios, y gestión de credenciales segura para el acceso entre servicios (ej. AWS IAM Roles, Azure Managed Identities).
    b. Justificación: Garantiza que solo los usuarios y servicios autorizados puedan acceder a los recursos y realizar acciones específicas, minimizando el riesgo de accesos privilegiados indebidos.

4. Monitoreo de Seguridad y Detección de Intrusiones (IDS/IPS):
    a. Herramientas: Implementación de SIEM, WAF (Web Application Firewall) en el API Gateway, IDS/IPS a nivel de red y escaneo de vulnerabilidades continuo.
    b. Justificación: Permite la detección temprana de actividades sospechosas, ataques cibernéticos y vulnerabilidades, facilitando una respuesta rápida y mitigación de riesgos.

5. Hardening de Componentes:
    a. Práctica: Configuración segura de sistemas operativos, contenedores, bases de datos y frameworks. Esto incluye eliminar servicios innecesarios, aplicar parches de seguridad regularmente y configurar firewalls.
    b. Justificación: Reduce la superficie de ataque y minimiza las vulnerabilidades conocidas.

7. Alta Disponibilidad (HA), Tolerancia a Fallos, Recuperación ante Desastres (DR) y Monitoreo

7.1. Alta Disponibilidad (HA) y Tolerancia a Fallos

Se propone desplegar la arquitectura en una infraestructura de nube (Azure o AWS) que soporte múltiples zonas de disponibilidad (Availability Zones - AZ) dentro de una misma región.

- Despliegue Multi-AZ:

    - Justificación 1 (Tolerancia a Fallos de Zona): Al distribuir los componentes (microservicios, bases de datos) en al menos dos o tres Zonas de Disponibilidad, el sistema puede seguir operando incluso si una AZ completa experimenta una falla.
    - Justificación 2 (Reducción de Downtime): Proporciona redundancia activa/pasiva o activa/activa para los componentes críticos, minimizando el tiempo de inactividad durante fallos de hardware o software.

- Balanceadores de Carga (Load Balancers):
    - Implementación: Application Load Balancer (AWS ALBs) o Azure Application Gateway delante de los microservicios, distribuyendo el tráfico entre las instancias de cada servicio en diferentes AZs.
    - Justificación: Asegura una distribución equitativa de la carga, alta disponibilidad y la capacidad de enrutar el tráfico solo a instancias saludables (mediante health checks).

- Auto-Scaling (Escalado Automático):
    - Implementación: Grupos de Auto Scaling (AWS Auto Scaling Groups) o Conjuntos de Escala de Máquinas Virtuales (Azure Virtual Machine Scale Sets) para los microservicios.
    - Justificación: Permite que los servicios escalen horizontalmente (añadiendo o quitando instancias) automáticamente en respuesta a la demanda de tráfico, manteniendo el rendimiento y la disponibilidad durante picos de uso.

- Bases de Datos con Alta Disponibilidad:
    - Implementación: Utilizar servicios de bases de datos gestionados en la nube con réplicas de lectura, multi-AZ y failover automático (ej. AWS Aurora, Azure SQL Database/PostgreSQL Flexible Server con Geo-redundancia).
    - Justificación: Garantiza la durabilidad de los datos y la disponibilidad continua de la base de datos, con failover automático a una réplica en caso de fallo de la instancia primaria.

- Contenedores y Orquestación (Kubernetes - EKS/AKS):
    - Implementación: Desplegar los microservicios como contenedores gestionados por Kubernetes (AWS EKS o Azure AKS).
    - Justificación: Kubernetes proporciona resiliencia inherente al orquestar y gestionar los contenedores, reubicando automáticamente los pods fallidos a nodos saludables y asegurando el número deseado de réplicas en ejecución.

7.2. Recuperación ante Desastres (DR)

Se recomienda una estrategia de Disaster Recovery con RPO bajo y RTO bajo (punto de recuperación y tiempo de recuperación objetivo), idealmente un modelo Active-Passive o Active-Active en múltiples regiones geográficas.

- Estrategia Multi-Región:
    - Implementación: Desplegar una instancia "piloto ligero" (Pilot Light) o una "multiregional activa/pasiva" (Warm Standby) de la infraestructura en una segunda región geográfica de la nube.
    - Justificación 1 (Protección contra Desastres Regionales): Protege el sistema contra fallos catastróficos que afecten a una región completa.
        - Justificación 2 (RTO/RPO): Permite una rápida recuperación con una pérdida mínima de datos y un tiempo de inactividad reducido.

    - Replicación de Datos: Replicación continua y asíncrona de las bases de datos entre regiones (ej. AWS Aurora Global Database, Azure Geo-Replication para Cosmos DB/SQL DB).

- Backups y Snapshots:
    - Implementación: Políticas automáticas de backup para todas las bases de datos y volúmenes de almacenamiento, con replicación de backups a la región de DR.
    - Justificación: Es el último recurso para la recuperación de datos en caso de corrupción o pérdida, complementando las estrategias de HA y DR.

7.3. Monitoreo y Observabilidad

Un sistema bancario requiere monitoreo exhaustivo para garantizar la operatividad y la detección temprana de problemas.

- Monitoreo de Infraestructura y Aplicaciones (APM):
    - Herramientas: AWS CloudWatch, Azure Monitor, Prometheus/Grafana, Datadog, New Relic.
    - Justificación: Recopila métricas sobre CPU, memoria, red, disco, uso de bases de datos, latencia de solicitudes, errores y rendimiento de cada microservicio.

- Logging Centralizado:
    - Herramientas: AWS CloudWatch Logs, Azure Log Analytics, ELK Stack (Elasticsearch, Logstash, Kibana), Splunk.
    - Justificación: Agrega logs de todas las aplicaciones y la infraestructura en un solo lugar.

- Alertas y Notificaciones:
    - Implementación: Configuración de alertas en función de umbrales críticos (ej. alta latencia, errores 5xx, baja disponibilidad de instancias) que se envían a equipos de operación a través de Slack, PagerDuty, email, etc.
    - Justificación: Permite una respuesta proactiva a incidentes, minimizando el impacto en los usuarios.

- Distributed Tracing (Trazabilidad Distribuida):
    - Herramientas: Jaeger, Zipkin, AWS X-Ray, Azure Application Insights.
    - Justificación: Permite rastrear la ruta de una solicitud a través de múltiples microservicios.

- Health Checks y Auto-Healing:
    - Implementación: Los orquestadores de contenedores (Kubernetes) realizan health checks continuos en los pods. Si un pod no responde, es automáticamente reiniciado o reemplazado.
    - Justificación: Proporciona capacidades de auto-recuperación, reduciendo la necesidad de intervención manual.

8. Arquitectura Desacoplada y Reutilizable

La arquitectura propuesta está diseñada para ser altamente desacoplada, con componentes reutilizables.

- Principios de Diseño:
    - Single Responsibility Principle (SRP): Cada microservicio tiene una única responsabilidad bien definida.
    - Loose Coupling, High Cohesion: Los microservicios están débilmente acoplados entre sí.

- API-First Design: Todas las interacciones entre servicios se realizan a través de APIs RESTful bien documentadas, lo que permite la evolución independiente de los servicios y facilita la integración con sistemas externos o futuros componentes.

- Event-Driven Architecture (EDA): Para ciertos flujos (ej. notificaciones, auditoría), se pueden utilizar colas de mensajes (Kafka, RabbitMQ) para comunicar eventos de forma asíncrona.
- Justificación: El EDA reduce aún más el acoplamiento entre servicios. Un servicio puede publicar un evento sin saber qué otros servicios lo consumirán.

- Elementos Reutilizables:
  - API Gateway: Proporciona un punto de entrada unificado y centraliza la gestión de seguridad y enrutamiento.
  - Servicio de Autenticación: Al ser un componente independiente que implementa OAuth2.0, puede ser reutilizado por cualquier otra aplicación o servicio de BP.
  - Servicio de Auditoría: Un servicio centralizado para registrar eventos de auditoría es reutilizable.
  - Orquestador de Notificaciones: Centraliza la lógica de envío de notificaciones y puede ser utilizado por cualquier otro sistema interno.

9. Manejo de Costos (Consideraciones en Nube)

La elección de una infraestructura en la nube (Azure o AWS) ofrece flexibilidad y herramientas para gestionar los costos de manera eficiente.

- Modelo de Pago por Uso (Pay-as-you-go):
  - Justificación: Se paga solo por los recursos consumidos, lo que es ideal para la variabilidad de la demanda en un sistema bancario.

- Instancias Reservadas / Planes de Ahorro:
  - Justificación: Para cargas de trabajo predecibles y sostenidas (ej. bases de datos, microservicios base que siempre están activos), la compra de instancias reservadas (RI) o el uso de Savings Plans (AWS) / Azure Reserved Instances puede generar ahorros significativos.

- Auto-Scaling:
  - Justificación: Como se mencionó en HA, el escalado automático asegura que solo se paga por los recursos necesarios en un momento dado.

- Servicios Gestionados (Managed Services):
  - Justificación: Utilizar servicios de bases de datos gestionados (RDS, Aurora, Cosmos DB), Kubernetes gestionado (EKS, AKS), API Gateway gestionado, etc., reduce la sobrecarga operativa de gestión de infraestructura.

- Optimización de Recursos:
  - Monitoreo de Costos: Utilizar herramientas de gestión de costos de la nube (AWS Cost Explorer, Azure Cost Management) para identificar patrones de gasto, recursos infrautilizados y oportunidades de optimización.
  - Right-Sizing: Ajustar el tamaño de las instancias y los recursos de la base de datos a la demanda real, evitando el exceso de aprovisionamiento.

    - Contenedores y Serverless (Lambda/Azure Functions): Los contenedores son más eficientes en el uso de recursos que las VMs tradicionales, y las funciones serverless pueden reducir aún más los costos al pagar solo por el tiempo de ejecución.

- Estrategia de Almacenamiento:
    - Justificación: Elegir el tipo de almacenamiento adecuado para cada necesidad (ej. S3 Infrequent Access/Glacier para backups y auditoría de largo plazo, SSDs de alto rendimiento para bases de datos transaccionales) optimiza los costos sin comprometer el rendimiento donde es crítico.